Step 4 — A Real Client That Connects and Uses All Three — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Two hints — work through them in order.

- Hint 1 — The pieces, and where each one lives
- Hint 2 — The plan, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The pieces, and where each one lives

One new file, `client.py`, at the repo root. `server.py` does not change.

- Launch the server as a subprocess: `StdioServerParameters(command="python", args=["server.py"])`, then `stdio_client(...)` and a `ClientSession`, and `await session.initialize()`.
- **Discover, don't hardcode:** `await session.list_tools()`, `list_resources()`, `list_prompts()`. Your client never imports anything from `server.py` — it asks the server what it offers (Doc06's "discovered, not hardcoded").
- **Use each one:** `call_tool("add_note", {...})`, `call_tool("search_notes", {...})`, `read_resource("notes://all")`, `get_prompt("summarize_notes", {...})`.
- Wrap the whole connection in `try/except` so a missing `server.py` prints one clear line, not a raw async traceback.

Two SDK details worth knowing: a tool result's text is `result.content[0].text`; a resource's is `result.contents[0].text`; a prompt's is `result.messages[0].content.text`. And the error flag is `result.is_error` (snake_case) in the current SDK — but since Step 2's server returns a clear `"Error: ..."` string for a rejected note, checking `text.startswith("Error")` is the simplest, version-independent test.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
# client.py
import asyncio

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

async def main() -> None:
    server_params = StdioServerParameters(command="python", args=["server.py"])
    try:
        async with stdio_client(server_params) as (read, write):
            async with ClientSession(read, write) as session:
                await session.initialize()
                # your turn: list_tools/list_resources/list_prompts and print
                # each (build the name lists with plain for-loops, not a
                # comprehension); then call_tool add_note + search_notes,
                # read_resource notes://all, get_prompt summarize_notes;
                # finally call add_note with "   " and confirm the result
                # text starts with "Error"
                ...
    except Exception as e:
        print(f"Could not start or connect to the notes server: {e}")

if __name__ == "__main__":
    asyncio.run(main())
```

Run `python client.py` and confirm all four things succeed in one run. Then rename `server.py` temporarily and rerun — you should get the one clear "Could not start or connect" line, not a traceback. Compare against the Solution.